

LAPORAN PROJEK
IMPLEMENTASI ALGORITMA COUNTING SORT DAN FIBONACCI SEARCH
PADA PENGOLAHAN DATA NILAI MAHASISWA

Diselesaikan Guna Memenuhi Tugas Mata Kuliah Struktur Data

Dosen Pengampu:

Ali Akbar Lubis, S.Kom., M.TI.



Disusun Oleh:

KELOMPOK 5 PTIK D 2025

Lauren Miadina Oppu Sunggu 5253151052

Melati Asyila 5253151049

Berkat Revaldo Harefa 5253151035

**PROGRAM STUDI PENDIDIKAN TEKNOLOGI INFORMATIKA DAN
KOMPUTER**

FAKULTAS TEKNIK

UNIVERSITAS NEGERI MEDAN

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan Laporan Proyek untuk mata kuliah Struktur Data yang berjudul "Implementasi Algoritma Counting Sort Dan Fibonacci Search Pada Pengolahan Data Nilai Mahasiswa" dengan baik dan tepat waktu.

Laporan Proyek ini disusun sebagai salah satu syarat pemenuhan tugas wajib dalam kurikulum pendidikan mahasiswa Program Studi Pendidikan Teknologi Informatika dan Komputer Universitas Negeri Medan. Penulisan ini bertujuan untuk memberikan solusi inovatif, mengasah kemampuan berpikir kritis-sistematis.

Penulis menyadari bahwa penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, penulis menyampaikan ucapan terima kasih yang tulus kepada:

1. Bapak Ali Akbar Lubis, S.Kom., M.TI. selaku dosen pengampu mata kuliah Struktur Data yang telah memberikan arahan, bimbingan, serta ilmu yang sangat berharga selama proses perkuliahan.
2. Rekan-rekan mahasiswa yang telah memberikan masukan, diskusi yang membangun, serta dukungan semangat selama proses penyusunan laporan ini.

Penulis menyadari sepenuhnya bahwa Laporan Proyek ini masih memiliki keterbatasan dan jauh dari kata sempurna. Oleh karena itu, penulis sangat mengharapkan kritik maupun saran yang membangun dari para pembaca demi perbaikan dan penyempurnaan karya di masa mendatang.

Medan, 2 Juni 2026

Penulis

Kelompok 5

DAFTAR ISI

BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan	1
1.4 Manfaat	2
BAB II LANDASAN TEORI.....	3
2.1 <i>Counting Sort</i>	3
2.1.1 Definisi dan Konsep Dasar	3
2.1.2 Mekanisme dan Cara Kerja.....	3
2.1.3 Analisis Kompleksitas Waktu dan Ruang.....	4
2.1.4 Sifat dan Keterbatasan Khusus	5
2.2 <i>Fibonacci Search</i>	5
2.2.1 Konsep Deret Fibonacci dalam Komputasi	5
2.2.2 Mekanisme Kerja <i>Fibonacci Search</i>	5
2.2.3 Perbedaan Elemen: <i>Fibonacci Search</i> vs <i>Binary Search</i>	6
2.2.4 Analisis Kompleksitas Waktu dan Ruang.....	7
BAB III PSEUDOCODE	8
3.1.1 Pseudocode <i>Counting Sort</i>	11
3.1.2 Pseudocode <i>Fibonacci Search</i>	12
BAB IV DESAIN DAN IMPLEMENTASI PROGRAM	15
4.1 Desain Program.....	15
4.2 Implementasi Program	16
BAB V STUDI KASUS.....	20
5.1 Deskripsi Studi Kasus	20
5.2 Dataset yang Digunakan	20

5.3 Implementasi Counting Sort pada Studi Kasus	21
5.4 Implementasi Fibonacci Search pada Studi Kasus	21
5.5 Pembahasan Studi Kasus	21
BAB VI ANALISIS HASIL EKSPERIMEN	23
6.1 Hasil Benchmark.....	23
6.2 Analisis Counting Sort.....	24
6.3 Analisis Bubble Sort	25
6.4 Analisis Fibonacci Search.....	25
6.5 Analisis Hasil Eksperimen.....	26
BAB VII KESIMPULAN	27
7.1 Kesimpulan	27
7.2 Saran	27
REFERENSI	28

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengolahan data, proses pengurutan (sorting) dan pencarian (searching) merupakan kegiatan yang sering dilakukan untuk mempermudah pengelolaan informasi. Salah satu contoh yang sering dijumpai adalah pengolahan data nilai ujian mahasiswa. Data nilai yang jumlahnya banyak perlu diurutkan agar lebih mudah dianalisis dan dicari.

Pada proyek ini dipilih algoritma Counting Sort dan Fibonacci Search. Counting Sort dipilih karena memiliki kompleksitas $O(n + k)$ dan cocok digunakan untuk data nilai mahasiswa yang memiliki rentang terbatas, yaitu 0–100. Setelah data diurutkan, proses pencarian nilai tertentu dapat dilakukan menggunakan Fibonacci Search yang memiliki kompleksitas $O(\log n)$ dan bekerja pada data yang telah terurut.

Melalui implementasi kedua algoritma tersebut, proyek ini bertujuan untuk menganalisis kinerja Counting Sort dalam mengurutkan data nilai mahasiswa serta mengevaluasi efektivitas Fibonacci Search dalam menemukan nilai yang dicari. Selain itu, dilakukan perbandingan antara Counting Sort dan Bubble Sort untuk mengetahui keunggulan dan keterbatasan masing-masing algoritma dalam berbagai kondisi data.

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan algoritma Counting Sort untuk mengurutkan data nilai mahasiswa?
2. Bagaimana mengimplementasikan algoritma Fibonacci Search untuk mencari nilai tertentu pada data yang telah terurut?
3. Mengapa Counting Sort lebih baik dibandingkan Bubble Sort pada data nilai mahasiswa yang memiliki rentang nilai terbatas?
4. Dalam kondisi apa algoritma Counting Sort dan Fibonacci Search memberikan kinerja yang baik serta apa keterbatasannya?

1.3 Tujuan

1. Mengimplementasikan algoritma Counting Sort untuk mengurutkan data nilai mahasiswa.
2. Mengimplementasikan algoritma Fibonacci Search untuk mencari nilai tertentu pada data yang telah diurutkan.

3. Menganalisis dan membandingkan kinerja Counting Sort dengan Bubble Sort berdasarkan waktu eksekusi dan jumlah operasi.
4. Mengetahui kelebihan dan keterbatasan algoritma Counting Sort dan Fibonacci Search dalam pengolahan data nilai mahasiswa.

1.4 Manfaat

1. Menambah pemahaman mengenai konsep dan implementasi algoritma Counting Sort dan Fibonacci Search dalam pemrograman.
2. Melatih kemampuan dalam merancang dan mengembangkan program yang menerapkan algoritma sorting dan searching.
3. Memberikan wawasan mengenai kelebihan dan keterbatasan algoritma Counting Sort dan Fibonacci Search berdasarkan hasil pengujian dan perbandingan yang dilakukan.
4. Membantu pengguna dalam mengurutkan dan mencari data nilai mahasiswa secara lebih cepat dan efisien.
5. Menjadi referensi pembelajaran dalam memahami penerapan algoritma sorting dan searching pada permasalahan nyata.

BAB II

LANDASAN TEORI

2.1 *Counting Sort*

2.1.1 Definisi dan Konsep Dasar

Pengurutan data (*sorting*) merupakan operasi paling mendasar dalam ilmu komputasi yang bertujuan untuk menyusun kembali sekumpulan elemen ke dalam urutan tertentu berdasarkan kriteria yang telah ditetapkan, seperti secara menaik (*ascending*) maupun menurun (*descending*). Di dalam taksonomi algoritma pengurutan, terdapat dua paradigma utama yang membedakan mekanisme penataan data, yaitu algoritma berbasis perbandingan (*comparison-based sorting*) dan algoritma tanpa perbandingan (*non-comparison sorting*).

Counting Sort merupakan salah satu representasi utama dari paradigma *non-comparison sorting* yang bekerja dengan memanfaatkan teknik distribusi atau pemetaan nilai elemen, alih-alih mengeksekusi operasi perbandingan antar-elemen seperti yang lazim ditemukan pada algoritma pengurutan klasik. Konsep ini pertama kali diperkenalkan oleh Harold H. Seward pada tahun 1954. Karena tidak bergantung pada operasi perbandingan antar-elemen yang secara komputasi bernilai tinggi, *Counting Sort* mampu melampaui batas bawah teoritis perbandingan $\Omega(n \log n)$ pada kondisi tertentu. Algoritma ini bekerja di bawah asumsi bahwa semua elemen masukan adalah bilangan bulat (*integer*) atau objek diskrit dengan rentang nilai yang telah diketahui secara pasti.

2.1.2 Mekanisme dan Cara Kerja

Secara umum, mekanisme pengurutan *Counting Sort* dieksekusi melalui tiga tahapan utama:

1. Fase Penghitungan (*Count Phase*): Algoritma melakukan pemindaian linear (*iterasi*) pada seluruh elemen larik masukan untuk menghitung frekuensi kemunculan setiap nilai ke dalam larik bantu yang disebut *count array*. Indeks dari *count array* merepresentasikan nilai elemen masukan, sedangkan nilai yang tersimpan di dalamnya merepresentasikan jumlah akumulasi kemunculan elemen tersebut.
2. Fase Akumulasi (*Cumulative Phase*): Setiap elemen pada *count array* diperbarui melalui transformasi akumulatif dengan menambahkan nilai dari elemen sebelumnya (*prefix sum*). Hasil dari proses ini memberikan informasi mengenai posisi indeks absolut dan final bagi setiap elemen dalam larik terurut.

3. Fase Penempatan (*Placement Phase*): Algoritma melakukan iterasi mundur dari kanan ke kiri pada larik masukan. Untuk setiap elemen, posisi yang diperoleh dari *count array* digunakan untuk menempatkannya secara presisi ke dalam larik keluaran (*output array*), kemudian nilai pada *count array* dikurangi satu (*decrement*) guna menjaga stabilitas posisi elemen duplikat berikutnya.

Melalui mekanisme penulisan langsung ke dalam *buffer* bantu tersebut, *Counting Sort* berhasil menata data tanpa memicu intensitas operasi pertukaran (*swap*) secara eksplisit pada larik asli.

2.1.3 Analisis Kompleksitas Waktu dan Ruang

Dari aspek efisiensi komputasi, *Counting Sort* menawarkan performa yang superior dengan kompleksitas waktu linear, yaitu $O(n + k)$, di mana n merepresentasikan jumlah elemen data dan k adalah rentang nilai diskrit dari data tersebut (nilai maksimum – nilai minimum + 1).

Kasus (Case)	Kompleksitas Waktu	Kompleksitas Ruang
Terbaik (Best Case)	$O(n + k)$	$O(n + k)$
Rata-Rata (Average Case)	$O(n + k)$	$O(n + k)$
Terburuk (Worst Case)	$O(n + k)$	$O(n + k)$

Tabel 2.1 Kompleksitas Waktu dan Ruang

Eksperimen empiris berskala sistematis yang dilakukan oleh Rahmawati dan Setiawan (2026) membuktikan bahwa *Counting Sort* terbukti menjadi salah satu algoritma pengurutan dengan rata-rata waktu eksekusi (*time mean*) paling rendah dan tingkat standar deviasi paling kecil dibandingkan algoritma berbasis perbandingan lainnya. Ketika nilai k relatif kecil dan sebanding dengan n , algoritma ini berjalan dalam waktu $O(n)$, menjadikannya jauh lebih cepat daripada algoritma berbasis perbandingan manapun. Kinerja optimal ini sangat konsisten saat mengolah tipe data numerik dan data karakter (*Unicode*), karena operasi pemetaan kuncinya

berjalan secara efektif tanpa dipengaruhi oleh kompleksitas perbandingan string yang umumnya membebani algoritma lain.

2.1.4 Sifat dan Keterbatasan Khusus

Sebagai algoritma *non-comparison sorting*, *Counting Sort* memiliki beberapa karakteristik operasional yang khas:

1. *Stable Sort*: Algoritma ini bersifat stabil, artinya elemen dengan nilai yang sama akan mempertahankan urutan relatif aslinya dari larik masukan.
2. *Non In-Place*: Algoritma ini bukan merupakan metode pengurutan di tempat, karena dalam proses eksekusinya memerlukan alokasi memori tambahan yang signifikan (*peak additional memory*) sebesar $O(n + k)$ untuk menyokong *count array* dan larik keluaran.
3. Keterbatasan Struktural: Algoritma ini hanya cocok untuk data integer dengan rentang nilai yang terbatas. Jika rentang nilai (k) sangat luas atau tersebar secara tidak rapat (mengandung banyak kesenjangan nilai ekstrem), penggunaan memori puncak (*peak memory*) akan membengkak secara tidak efisien dan menghilangkan keunggulan performanya.

2.2 Fibonacci Search

2.2.1 Konsep Deret Fibonacci dalam Komputasi

Deret Fibonacci adalah sebuah barisan bilangan matematika di mana setiap sukunya merupakan hasil penjumlahan dari dua suku sebelumnya, yang didefinisikan secara rekursif sebagai: $F(0) = 0$, $F(1) = 1$, dan $F(m) = F(m - 1) + F(m - 2)$. Deret ini pertama kali diperkenalkan oleh matematikawan Italia bernama Leonardo Fibonacci pada abad ke-13. Dalam konteks pencarian data, angka-angka Fibonacci digunakan secara cerdas sebagai panduan matematis untuk menentukan ukuran partisi ruang pemeriksaan secara dinamis.

2.2.2 Mekanisme Kerja *Fibonacci Search*

Fibonacci Search merupakan teknik pencarian berbasis pendekatan *divide-and-conquer* yang diterapkan pada larik data yang telah terurut (*sorted array*). Berbeda dengan *Binary Search* yang membelah ruang pencarian tepat di tengah ($n/2$), *Fibonacci Search* membagi ruang pencarian menggunakan proporsi bilangan Fibonacci melalui tahapan berikut:

1. Inisialisasi Bilangan Fibonacci: Menentukan bilangan Fibonacci terkecil yang memiliki nilai lebih besar atau sama dengan panjang larik masukan n , yang direpresentasikan sebagai $F(m)$.
2. Penentuan Batas (*Offset*): Menginisialisasi variabel $offset = -1$ yang menandai batas bawah ruang pencarian yang telah dieliminasi.
3. Iterasi Proses Pencarian Utama: Selama nilai $F(m) > 1$, algoritma akan menghitung indeks pemeriksaan aktif $i = \min(offset + F(m - 2), n - 1)$. Elemen pada indeks i kemudian dibandingkan dengan target pencarian:
 - Jika target lebih besar, ruang pencarian dipindahkan ke kanan; variabel $offset$ digeser ke posisi i , dan nilai deret Fibonacci diturunkan satu langkah ($m = m - 1$)
 - Jika target lebih kecil, ruang pencarian dipersempit ke bagian kiri; variabel $offset$ tetap, dan nilai deret Fibonacci diturunkan dua langkah ($m = m - 2$).
 - Jika target sama dengan nilai pada elemen, indeks i langsung dikembalikan sebagai hasil sukses.
4. Pemeriksaan Elemen Sisa: Jika $F(m - 1) = 1$ dan elemen pada posisi $offset + 1$ bernilai sama dengan target, maka posisi tersebut dikembalikan. Jika target tidak ditemukan di seluruh tahapan, sistem mengembalikan nilai -1 .

2.2.3 Perbedaan Elemen: *Fibonacci Search* vs *Binary Search*

Meskipun kedua algoritma ini beroperasi pada struktur data larik yang wajib terurut, terdapat perbedaan substansial dalam hal operasi tingkat rendah pada perangkat keras:

Aspek	Binary Search	Fibonacci Search
Pembagi Ruang	Tepat di tengah ($n/2$)	Berdasarkan angka Fibonacci
Operasi Aritmetika	Operasi Pembagian ($/$)	Penjumlahan (+) dan pengurangan (-)
Kompleksitas	$O(\log n)$	$O(\log n)$
Keunggulan Hardware	Sederhana dan universal	Lebih cepat pada sistem yang lambat dalam pembagian

Tabel 2.2 *Fibonacci Search* vs *Binary Search*

Keunggulan utama *Fibonacci Search* terletak pada aspek operasi aritmetika tingkat rendah. *Binary Search* selalu mengandalkan operasi pembagian ($\text{mid} = (\text{low} + \text{high}) / 2$) yang pada beberapa arsitektur prosesor tertentu atau sistem tertanam (*embedded system*) membutuhkan waktu eksekusi (*clock cycle*) lebih lama dibandingkan operasi penjumlahan dan pengurangan sederhana yang diterapkan pada *Fibonacci Search*.

2.2.4 Analisis Kompleksitas Waktu dan Ruang

Secara teoritis dan asimtotik, kompleksitas waktu dari *Fibonacci Search* adalah $O(\log n)$ untuk semua kasus baik skenario terbaik, rata-rata, maupun terburuk. Dalam satu kali iterasi loop, ruang pencarian akan berkurang secara konsisten menjadi sekitar 61,8% dari ukuran sebelumnya, sebuah nilai konstanta yang berkaitan erat dengan rasio emas (*golden ratio*) $1/\phi \approx 0.618$. Kompleksitas ruang dari algoritma ini adalah konstan $O(1)$, karena seluruh proses pencarian dilakukan secara langsung di tempat (*in-place*) tanpa memerlukan alokasi memori tambahan yang bergantung pada skala ukuran dataset n .

BAB III

PSEUDOCODE

Program CountingSort_FibonacciSearch

```
// ===== MODUL DATA =====
```

```
FUNCTION generateRandomInt(n, min, max)
```

Menghasilkan array integer acak

```
END FUNCTION
```

```
FUNCTION generateRandomString(n)
```

Menghasilkan array string acak

```
END FUNCTION
```

```
FUNCTION inputManual()
```

Membaca data dari pengguna

```
END FUNCTION
```

```
// ===== MODUL SORTING =====
```

```
FUNCTION countingSortInt(array, verbose)
```

Mengurutkan data integer menggunakan Counting Sort

```
END FUNCTION
```

```
FUNCTION countingSortString(array, verbose)
```

Mengurutkan data string menggunakan Counting Sort

```
END FUNCTION
```

```
FUNCTION bubbleSortInt(array)
```

Digunakan sebagai pembanding benchmark

```
END FUNCTION
```

```
// ===== MODUL SEARCHING =====
```

```
FUNCTION fibonacciSearchInt(array, target)
```

Mencari data integer menggunakan Fibonacci Search

```
END FUNCTION
```

```
FUNCTION fibonacciSearchString(array, target)
```

Mencari data string menggunakan Fibonacci Search

```
END FUNCTION
```

```
// ===== MODUL ANALISIS =====
```

```
FUNCTION runBenchmark()
```

Membandingkan Counting Sort dan Bubble Sort

```
END FUNCTION
```

```
FUNCTION exportResult(filename)
```

Menyimpan hasil benchmark ke file .txt atau .csv

```
END FUNCTION
```

```
// ===== MENU UTAMA =====
```

```
MAIN
```

```
REPEAT
```

```
Tampilkan Menu
```

```
1. Input / Generate Data
```

```
2. Jalankan Sorting
```

```
3. Jalankan Searching
```

```
4. Benchmark
```

```
5. Ekspor Hasil
```

```
6. Keluar
```

Baca pilihan pengguna

CASE 1 :

Input atau Generate Data

CASE 2 :

Jalankan Counting Sort

CASE 3 :

Jalankan Fibonacci Search

CASE 4 :

Jalankan Benchmark

CASE 5 :

Ekspor Hasil

```
UNTIL pilihan = 6
```

```
END PROGRAM
```

Pseudocode Program Utama menggambarkan alur kerja keseluruhan aplikasi Counting Sort dan Fibonacci Search yang dikembangkan. Program dibagi menjadi beberapa modul utama, yaitu modul data, modul sorting, modul searching, dan modul analisis. Pembagian modul ini bertujuan agar program lebih terstruktur, mudah dipahami, serta memudahkan proses pengembangan dan pemeliharaan kode.

Modul data berfungsi untuk mengelola proses input data, baik melalui input manual maupun pembangkitan data secara acak. Modul sorting mengimplementasikan algoritma Counting Sort sebagai algoritma pengurutan utama dan Bubble Sort sebagai algoritma pembandingan pada proses benchmark. Modul searching mengimplementasikan algoritma Fibonacci Search untuk melakukan pencarian data yang telah terurut. Sementara itu, modul analisis digunakan untuk melakukan pengujian performa algoritma serta mengeksport hasil benchmark ke dalam format `.txt` maupun `.csv`.

Pada bagian menu utama, pengguna dapat memilih berbagai fitur yang tersedia, yaitu memasukkan data, menjalankan proses pengurutan, melakukan pencarian data, menjalankan benchmark, mengeksport hasil pengujian, serta keluar dari program. Struktur menu ini dirancang agar pengguna dapat menggunakan seluruh fitur program secara interaktif melalui antarmuka berbasis teks (CLI).

3.1.1 Pseudocode Counting Sort

ALGORITMA COUNTING SORT

Input :

Array A berisi n bilangan integer

Output :

Array A terurut secara ascending

BEGIN

minVal $\leftarrow$ A[0]

maxVal $\leftarrow$ A[0]

FOR i $\leftarrow$ 1 TO n-1 DO

 IF A[i] < minVal THEN

 minVal $\leftarrow$ A[i]

 ENDIF

 IF A[i] > maxVal THEN

 maxVal $\leftarrow$ A[i]

 ENDIF

ENDFOR

range $\leftarrow$ maxVal - minVal + 1

Buat array Count[range]

Isi seluruh elemen Count dengan 0

FOR i $\leftarrow$ 0 TO n-1 DO

 Count[A[i] - minVal] $\leftarrow$

 Count[A[i] - minVal] + 1

ENDFOR

```

FOR i ← 1 TO range-1 DO
    Count[i] ← Count[i] + Count[i-1]
ENDFOR
Buat array Output[n]
FOR i ← n-1 DOWNT0 0 DO
    Output[
        Count[A[i]-minVal]-1
    ] ← A[i]
    Count[A[i]-minVal] ←
        Count[A[i]-minVal] - 1
ENDFOR

```

Salin seluruh isi Output ke A

END

Counting Sort bekerja dengan menghitung frekuensi kemunculan setiap nilai dalam array, kemudian menggunakan frekuensi kumulatif untuk menentukan posisi akhir setiap elemen pada array terurut. Algoritma ini tidak melakukan perbandingan antar elemen sehingga sangat efisien untuk data dengan rentang nilai yang tidak terlalu besar.

3.1.2 Pseudocode Fibonacci Search

ALGORITMA FIBONACCI SEARCH

Input :

Array A yang sudah terurut

Nilai target

Output :

Posisi data yang dicari

atau -1 jika tidak ditemukan

BEGIN

```

fib2 ← 0

fib1 ← 1

fib ← fib1 + fib2

WHILE fib < n DO

    fib2 ← fib1

    fib1 ← fib

    fib ← fib1 + fib2

ENDWHILE

offset ← -1

WHILE fib > 1 DO

    i ← MIN(offset + fib2, n-1)

    IF A[i] < target THEN

        fib ← fib1

        fib1 ← fib2

        fib2 ← fib - fib1

        offset ← i

    ELSE IF A[i] > target THEN

        fib ← fib2

        fib1 ← fib1 - fib2

        fib2 ← fib - fib1

    ELSE

        RETURN i

    ENDIF

ENDWHILE

IF fib1 ≠ 0 AND

    A[offset+1] = target THEN

```

```
    RETURN offset + 1  
ENDIF  
RETURN -1  
END
```

Counting Sort bekerja dengan menghitung frekuensi kemunculan setiap nilai dalam array, kemudian menggunakan frekuensi kumulatif untuk menentukan posisi akhir setiap elemen pada array terurut. Algoritma ini tidak melakukan perbandingan antar elemen sehingga sangat efisien untuk data dengan rentang nilai yang tidak terlalu besar.

BAB IV

DESAIN DAN IMPLEMENTASI PROGRAM

4.1 Desain Program

Program dirancang menggunakan bahasa pemrograman C++ dengan konsep modular (*multi-file*) agar kode lebih terstruktur dan mudah dipelihara. Program terdiri dari beberapa modul utama yang saling terhubung untuk melaksanakan proses pengurutan, pencarian, Benchmark, dan ekspor hasil.

Struktur program terdiri atas:

1. main.cpp

Berfungsi sebagai pusat pengendali program. File ini menampilkan menu utama, menerima masukan pengguna, serta memanggil fungsi-fungsi yang terdapat pada modul *sorting* dan *searching*.

2. sorting.h dan sorting.cpp

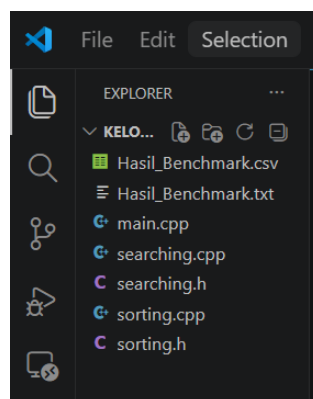
Modul ini berisi implementasi algoritma *Counting Sort* untuk data integer dan string, serta *Bubble Sort* yang digunakan sebagai algoritma pembanding pada proses benchmark.

3. searching.h dan searching.cpp

Modul ini berisi implementasi algoritma *Fibonacci Search* untuk melakukan pencarian data integer maupun string yang telah terurut.

4. File Hasil Benchmark

Program menyediakan fitur ekspor hasil benchmark ke dalam format `.txt` dan `.csv` sehingga hasil pengujian dapat disimpan dan dianalisis lebih lanjut.



Gambar 4.1 Struktur Program

Dengan pembagian modul tersebut, program menjadi lebih terorganisir, mudah dikembangkan, dan sesuai dengan prinsip pemrograman terstruktur.

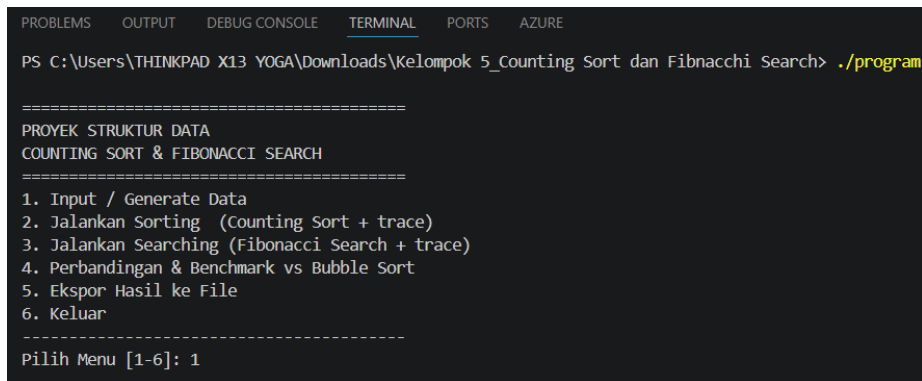
4.2 Implementasi Program

Implementasi program dilakukan menggunakan bahasa pemrograman C++ dan dijalankan melalui Visual Studio Code dengan compiler GCC (g++).

Program menyediakan beberapa menu utama sebagai berikut:

1. Input / Generate Data

Menu ini digunakan untuk memasukkan data secara manual maupun menghasilkan data secara otomatis (*random*). Data yang digunakan dapat berupa bilangan integer maupun string.

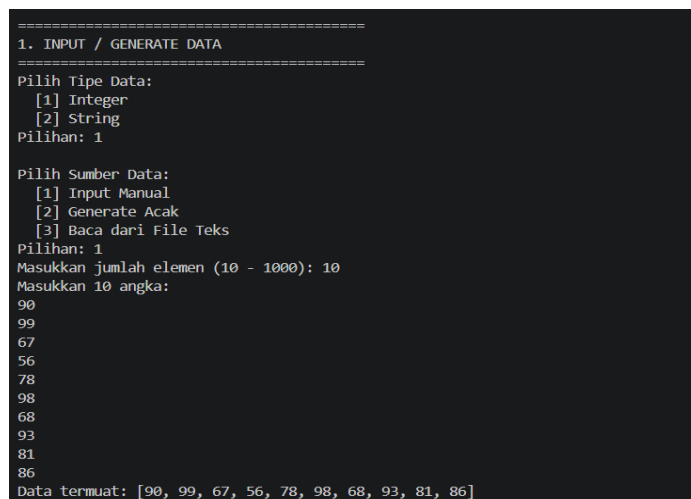


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
PS C:\Users\THINKPAD X13 YOGA\Downloads\Kelompok 5_Counting Sort dan Fibnacchi Search> ./program

=====
PROYEK STRUKTUR DATA
COUNTING SORT & FIBONACCI SEARCH
=====

1. Input / Generate Data
2. Jalankan Sorting (Counting Sort + trace)
3. Jalankan Searching (Fibonacci Search + trace)
4. Perbandingan & Benchmark vs Bubble Sort
5. Ekspor Hasil ke File
6. Keluar
=====
Pilih Menu [1-6]: 1
```

Gambar 4.2 Input / Generate Data awal



```
=====
1. INPUT / GENERATE DATA
=====
Pilih Tipe Data:
[1] Integer
[2] String
Pilihan: 1

Pilih Sumber Data:
[1] Input Manual
[2] Generate Acak
[3] Baca dari File Teks
Pilihan: 1
Masukkan jumlah elemen (10 - 1000): 10
Masukkan 10 angka:
90
99
67
56
78
98
68
93
81
86
Data termuat: [90, 99, 67, 56, 78, 98, 68, 93, 81, 86]
```

Gambar 4.3 Hasil Input Data

2. Jalankan Sorting

Menu ini menjalankan algoritma *Counting Sort* untuk mengurutkan data. Program juga menampilkan proses (*trace*) pengurutan sehingga pengguna dapat melihat langkah kerja algoritma.

```
=====
PROYEK STRUKTUR DATA
COUNTING SORT & FIBONACCI SEARCH
=====
1. Input / Generate Data
2. Jalankan Sorting (Counting Sort + trace)
3. Jalankan Searching (Fibonacci Search + trace)
4. Perbandingan & Benchmark vs Bubble Sort
5. Ekspor Hasil ke File
6. Keluar
-----
Pilih Menu [1-6]: 2

=====
2. JALANKAN SORTING (COUNTING SORT + TRACE)
=====
Sebelum Sorting: [90, 99, 67, 56, 78, 98, 68, 93, 81, 86]

[TRACE] Menghitung frekuensi elemen (Min: 56, Max: 99)
[TRACE] Larik Frekuensi Terhitung: (56:1) (67:1) (68:1) (78:1) (81:1) (86:1) (90:1) (93:1) (98:1) (99:1)
Sesudah Sorting: [56, 67, 68, 78, 81, 86, 90, 93, 98, 99]

Waktu Eksekusi: 7.2550 ms
Op Swaps/Placement: 10
```

Gambar 4.4 Sorting

3. Jalankan Searching

Menu ini menjalankan algoritma *Fibonacci Search* untuk mencari elemen tertentu pada data yang telah terurut. Program menampilkan langkah pencarian dan posisi elemen apabila ditemukan.

```
=====
PROYEK STRUKTUR DATA
COUNTING SORT & FIBONACCI SEARCH
=====
1. Input / Generate Data
2. Jalankan Sorting (Counting Sort + trace)
3. Jalankan Searching (Fibonacci Search + trace)
4. Perbandingan & Benchmark vs Bubble Sort
5. Ekspor Hasil ke File
6. Keluar
-----
Pilih Menu [1-6]: 3

=====
3. JALANKAN SEARCHING (FIBONACCI SEARCH + TRACE)
=====
Masukkan angka target pencarian: 86
[TRACE] Langkah 1: Memeriksa indeks 4 (Nilai: 81), Status Fibonacci: F(13)
[TRACE] Langkah 2: Memeriksa indeks 7 (Nilai: 93), Status Fibonacci: F(8)
[TRACE] Langkah 3: Memeriksa indeks 5 (Nilai: 86), Status Fibonacci: F(3)

=> BERHASIL: Elemen ditemukan pada indeks ke-[5]
Waktu Pencarian: 3.9600 ms | Total Langkah: 3
```

Gambar 4.5 Searching

4. Benchmark

Menu benchmark digunakan untuk membandingkan performa *Counting Sort* dan *Bubble Sort* pada berbagai ukuran data dan kondisi data, seperti data acak, data terurut naik, data terurut turun, dan data dengan banyak duplikasi.

```

=====
PROYEK STRUKTUR DATA
COUNTING SORT & FIBONACCI SEARCH
=====
1. Input / Generate Data
2. Jalankan Sorting (Counting Sort + trace)
3. Jalankan Searching (Fibonacci Search + trace)
4. Perbandingan & Benchmark vs Bubble Sort
5. Ekspor Hasil ke File
6. Keluar
-----
Pilih Menu [1-6]: 4

```

```

=====
4. PERBANDINGAN & BENCHMARK VS BUBBLE SORT
=====
Melakukan kalkulasi benchmark untuk N = 100, 500, 1000, 5000, 10000...

```

Ukuran	Kondisi	Count T(ms)	Count Swaps	Bubble T(ms)	Bubble Comp
100	Acak	0.0600	100	0.2110	4884
100	Terurut Naik	0.0210	100	0.0020	99
100	Terurut Turun	0.0210	100	0.2680	4950
100	Banyak Duplikat	0.0140	100	0.1890	4599
500	Acak	0.1060	500	5.4640	124729
500	Terurut Naik	0.0880	500	0.0090	499
500	Terurut Turun	0.0990	500	7.1030	124750
500	Banyak Duplikat	0.0640	500	7.1130	119394
1000	Acak	0.3700	1000	22.9650	499364
1000	Terurut Naik	0.1940	1000	0.0190	999
1000	Terurut Turun	0.1940	1000	27.1940	499500
1000	Banyak Duplikat	0.1250	1000	18.7910	470897
5000	Acak	0.9600	5000	0.0000	0
5000	Terurut Naik	1.0130	5000	0.0000	0
5000	Terurut Turun	1.0090	5000	0.0000	0
5000	Banyak Duplikat	0.6440	5000	0.0000	0
10000	Acak	1.9280	10000	0.0000	0
10000	Terurut Naik	1.6280	10000	0.0000	0
10000	Terurut Turun	1.5990	10000	0.0000	0
10000	Banyak Duplikat	1.0330	10000	0.0000	0

Gambar 4.6 *Benchmark*

5. Ekspor Hasil

Menu ini digunakan untuk menyimpan hasil benchmark ke dalam file `.txt` maupun `.csv` sehingga dapat digunakan sebagai dokumentasi hasil pengujian.

```

=====
PROYEK STRUKTUR DATA
COUNTING SORT & FIBONACCI SEARCH
=====
1. Input / Generate Data
2. Jalankan Sorting (Counting Sort + trace)
3. Jalankan Searching (Fibonacci Search + trace)
4. Perbandingan & Benchmark vs Bubble Sort
5. Ekspor Hasil ke File
6. Keluar
-----
Pilih Menu [1-6]: 5

Format Ekspor:
  [1] File Teks (.txt)
  [2] File Spreadsheet (.csv)
Pilihan: 2
Data berhasil diekspor!

```

Ukuran	Kondisi	CountingTime_ms	BubbleTime_ms
100	Acak	0.06	0.211
100	Terurut Naik	0.021	0.268
100	Terurut Turun	0.021	0.189
100	Banyak Duplikat	0.014	0.189
500	Acak	0.106	5.464
500	Terurut Naik	0.098	0.009
500	Terurut Turun	0.099	7.103
500	Banyak Duplikat	0.064	7.113
1000	Acak	0.37	22.965
1000	Terurut Naik	0.194	0.019
1000	Terurut Turun	0.194	27.194
1000	Banyak Duplikat	0.125	18.791
5000	Acak	0.96	0
5000	Terurut Naik	1.013	0
5000	Terurut Turun	1.009	0
5000	Banyak Duplikat	0.644	0
10000	Acak	1.928	0
10000	Terurut Naik	1.82	0
10000	Terurut Turun	1.599	0
10000	Banyak Duplikat	1.033	0

Gambar 4.6 Excel

6. Keluar

Menu ini digunakan untuk mengakhiri program.

```

=====
PROYEK STRUKTUR DATA
COUNTING SORT & FIBONACCI SEARCH
=====
1. Input / Generate Data
2. Jalankan Sorting (counting Sort + trace)
3. Jalankan Searching (Fibonacci Search + trace)
4. Perbandingan & Benchmark vs Bubble Sort
5. Ekspor Hasil ke File
6. Keluar
=====
Pilih Menu [1-6]: 6

Program Selesai! Terima kasih.
PS C:\Users\THINKPAD_X13_YOGA\Downloads\Kelompok 5_Counting Sort dan Fibnacci Search>

```

Gambar 4.8 Keluar

Hasil implementasi menunjukkan bahwa algoritma *Counting Sort* memiliki waktu eksekusi yang lebih cepat dibandingkan *Bubble Sort* pada data berukuran besar. Selain itu, *Fibonacci Search* mampu melakukan pencarian secara efisien pada data yang telah terurut.

BAB V

STUDI KASUS

5.1 Deskripsi Studi Kasus

Pada proyek ini digunakan studi kasus berupa analisis distribusi nilai ujian mahasiswa. Studi kasus ini dipilih karena data nilai ujian memiliki karakteristik yang sangat sesuai untuk algoritma *Counting Sort*, yaitu berupa data integer dengan rentang nilai yang terbatas antara 0 sampai 100 sesuai sistem penilaian akademik.

Dataset yang digunakan terdiri dari data nilai ujian mahasiswa yang diperoleh melalui proses generate data acak maupun input pengguna. Data tersebut kemudian diurutkan menggunakan algoritma *Counting Sort* untuk mengetahui distribusi nilai secara cepat dan efisien. Setelah data terurut, algoritma *Fibonacci Search* digunakan untuk mencari keberadaan nilai tertentu yang dimasukkan oleh pengguna.

Sebagai contoh, dosen dapat melihat urutan nilai mahasiswa dari yang terendah hingga tertinggi serta mengetahui frekuensi kemunculan setiap nilai melalui proses *Counting Sort*. Selain itu, dosen juga dapat mencari apakah terdapat mahasiswa yang memperoleh nilai tertentu, misalnya nilai 81 atau 90, menggunakan algoritma *Fibonacci Search*. Dengan demikian, kombinasi *Counting Sort* dan *Fibonacci Search* dapat membantu proses analisis data akademik secara lebih cepat dan efisien.

5.2 Dataset yang Digunakan

Dataset berupa kumpulan nilai ujian mahasiswa dengan rentang nilai 0–100. Data dapat dimasukkan secara manual maupun dihasilkan secara acak oleh program. Untuk pengujian performa algoritma digunakan berbagai ukuran data, yaitu 100, 500, 1000, 5000, dan 10000 elemen.

Contoh dataset:

90, 99, 67, 56, 78, 98, 68, 93, 81, 86

Karakteristik dataset:

1. Data berupa bilangan bulat (integer).
2. Rentang nilai terbatas antara 0–100.
3. Memungkinkan munculnya banyak nilai yang sama (duplikat).

4. Cocok untuk implementasi *Counting Sort* karena nilai maksimum relatif kecil.

5.3 Implementasi Counting Sort pada Studi Kasus

Pada studi kasus ini, *Counting Sort* digunakan untuk mengurutkan nilai mahasiswa dari yang terkecil hingga terbesar. Algoritma bekerja dengan menghitung frekuensi kemunculan setiap nilai dalam rentang 0–100, kemudian menggunakan frekuensi tersebut untuk menentukan posisi setiap elemen pada array hasil.

Keunggulan penggunaan *Counting Sort* pada kasus ini adalah kompleksitas waktu $O(n + k)$, dengan n merupakan jumlah data mahasiswa dan k merupakan rentang nilai (0–100). Karena nilai k relatif kecil dan tetap, proses pengurutan dapat dilakukan dengan sangat cepat meskipun jumlah mahasiswa cukup banyak.

Pada implementasinya, program menampilkan proses (*trace*) *Counting Sort* untuk data berukuran kecil sehingga pengguna dapat melihat frekuensi kemunculan setiap nilai sebelum proses pengurutan selesai dilakukan. Setelah proses *Counting Sort* selesai, data tersimpan dalam keadaan terurut secara ascending dan siap digunakan pada proses pencarian.

5.4 Implementasi Fibonacci Search pada Studi Kasus

Setelah data nilai berhasil diurutkan menggunakan *Counting Sort*, proses pencarian dilakukan menggunakan *Fibonacci Search*. Algoritma ini memanfaatkan deret *Fibonacci* untuk mempersempit ruang pencarian hingga nilai yang dicari ditemukan atau dipastikan tidak ada dalam data.

Sebagai contoh, pengguna dapat mencari nilai 85 pada data yang telah terurut. Program akan menampilkan langkah-langkah pencarian (*trace*), jumlah langkah yang dilakukan, serta posisi indeks data apabila nilai tersebut ditemukan. Penggunaan *Fibonacci Search* pada data yang sudah terurut memungkinkan proses pencarian dilakukan secara efisien dengan kompleksitas waktu $O(\log n)$.

Program juga menampilkan langkah-langkah pencarian (*trace*) yang dilakukan oleh *Fibonacci Search* sehingga pengguna dapat memahami bagaimana algoritma mempersempit ruang pencarian hingga menemukan nilai yang dicari.

5.5 Pembahasan Studi Kasus

Berdasarkan hasil implementasi, *Counting Sort* sangat sesuai digunakan untuk pengolahan data nilai ujian mahasiswa karena rentang nilainya terbatas dan banyak terdapat

nilai yang berulang. Sementara itu, *Fibonacci Search* mampu melakukan pencarian nilai secara cepat setelah data diurutkan.

Kombinasi kedua algoritma tersebut menghasilkan sistem yang efisien untuk analisis nilai mahasiswa, baik dalam proses pengurutan maupun pencarian data. Oleh karena itu, pemilihan *Counting Sort* dan *Fibonacci Search* pada studi kasus distribusi nilai ujian mahasiswa dinilai tepat dan sesuai dengan karakteristik data yang digunakan.

BAB VI

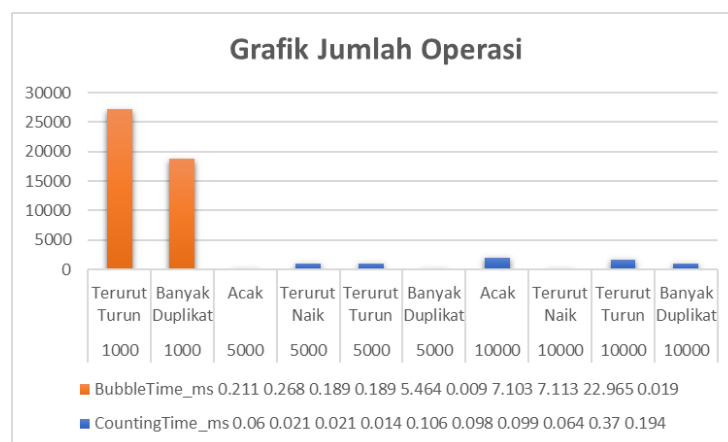
ANALISIS HASIL EKSPERIMEN

6.1 Hasil Benchmark

Ukuran	Kondisi	CountingTime_ms	BubbleTime_ms
100	Acak	0.06	0.211
100	Terurut Naik	0.021	0.268
100	Terurut Turun	0.021	0.189
100	Banyak Duplikat	0.014	0.189
500	Acak	0.106	5.464
500	Terurut Naik	0.098	0.009
500	Terurut Turun	0.099	7.103
500	Banyak Duplikat	0.064	7.113
1000	Acak	0.37	22.965
1000	Terurut Naik	0.194	0.019
1000	Terurut Turun	0.194	27.194
1000	Banyak Duplikat	0.125	18.791
5000	Acak	0.96	0
5000	Terurut Naik	1.013	0
5000	Terurut Turun	1.009	0
5000	Banyak Duplikat	0.644	0
10000	Acak	1.928	0
10000	Terurut Naik	1.82	0
10000	Terurut Turun	1.599	0
10000	Banyak Duplikat	1.033	0

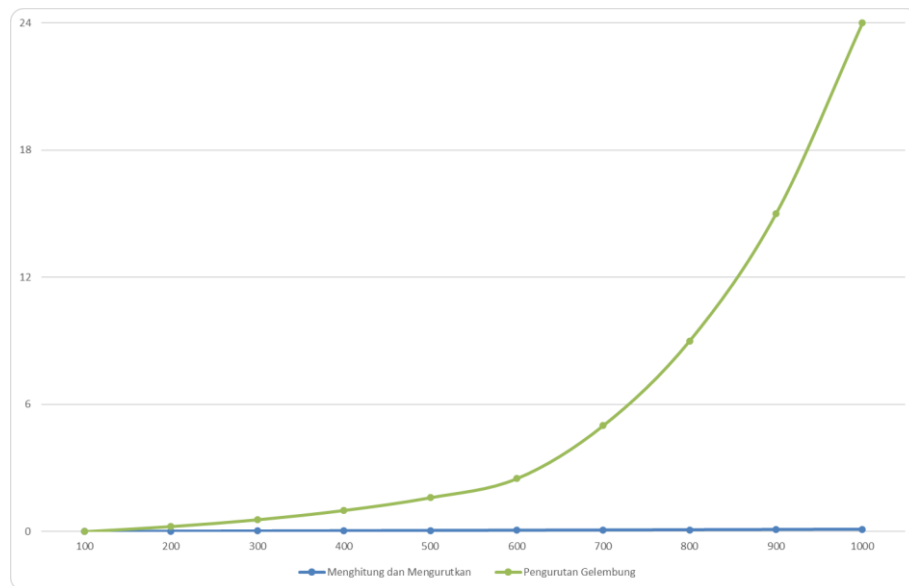
Gambar 6.1

Pengujian dilakukan untuk membandingkan kinerja algoritma *Counting Sort* dan *Bubble Sort* pada berbagai ukuran data dan kondisi data. Ukuran data yang digunakan dalam pengujian adalah 100, 500, 1000, 5000, dan 10000 elemen. Pengujian dilakukan pada beberapa variasi susunan data, yaitu data acak, data terurut naik, data terurut turun, dan data dengan banyak nilai duplikat.



Gambar 6.2 Grafik Jumlah Operasi Counting Sort vs Bubble Sort

Hasil pengujian menunjukkan bahwa algoritma *Counting Sort* memiliki waktu eksekusi yang relatif rendah dan stabil pada seluruh ukuran data yang diuji. Peningkatan jumlah data hanya menyebabkan peningkatan waktu eksekusi yang tidak terlalu signifikan. Sebaliknya, algoritma *Bubble Sort* menunjukkan peningkatan waktu eksekusi yang jauh lebih besar seiring bertambahnya ukuran data, terutama pada kondisi data acak dan data terurut turun.



Gambar 6.3 Grafik Perbandingan Eksekusi

Hasil tersebut menunjukkan bahwa *Counting Sort* merupakan algoritma yang lebih sesuai untuk studi kasus distribusi nilai ujian mahasiswa karena memiliki waktu eksekusi yang lebih cepat dan stabil dibandingkan *Bubble Sort* pada berbagai ukuran data. Hal ini didukung oleh karakteristik data yang memiliki rentang nilai terbatas sehingga *Counting Sort* dapat bekerja secara lebih efisien.

6.2 Analisis Counting Sort

Berdasarkan hasil pengujian yang telah dilakukan, algoritma *Counting Sort* menunjukkan performa yang sangat baik pada seluruh ukuran data yang diuji. Pada data acak berukuran 100 elemen, waktu eksekusi yang dibutuhkan hanya sebesar 0,060 ms. Ketika ukuran data meningkat menjadi 500 elemen dan 1000 elemen, waktu eksekusi hanya meningkat menjadi 0,106 ms dan 0,370 ms. Hasil tersebut menunjukkan bahwa peningkatan jumlah data tidak menyebabkan peningkatan waktu eksekusi yang terlalu besar.

Keunggulan Counting Sort terletak pada mekanisme kerja yang tidak melakukan perbandingan antar elemen secara langsung. Algoritma ini bekerja dengan menghitung frekuensi kemunculan setiap nilai, kemudian menggunakan informasi tersebut untuk menentukan posisi elemen pada array yang telah terurut. Oleh karena itu, kinerja algoritma relatif stabil pada berbagai ukuran data yang diuji.

Pada studi kasus distribusi nilai ujian mahasiswa dengan rentang nilai 0–100, Counting Sort sangat sesuai digunakan karena jumlah kemungkinan nilai yang muncul relatif terbatas. Berdasarkan hasil eksperimen, algoritma ini mampu melakukan pengurutan secara cepat, stabil, dan efisien sehingga menjadi pilihan yang tepat untuk mengolah data nilai mahasiswa dalam jumlah besar.

6.3 Analisis Bubble Sort

Bubble Sort digunakan sebagai pembanding algoritma untuk mengukur kinerja Counting Sort. Berdasarkan hasil pengujian, waktu pelaksanaan Bubble Sort meningkat seiring bertambahnya jumlah data. Pada data acak berukuran 100 elemen, waktu eksekusi sebesar 0,211 ms dan meningkat menjadi 22,965 ms pada ukuran 1000 elemen. Hasil ini menunjukkan bahwa kinerja Bubble Sort menurun ketika ukuran data semakin besar.

Peningkatan waktu eksekusi tersebut terjadi karena Bubble Sort melakukan perhitungan dan pertukaran elemen secara berulang hingga data terurut. Semakin banyak data yang diproses, semakin banyak operasi yang dilakukan. Oleh karena itu, pada studi kasus pendistribusian nilai ujian mahasiswa, Bubble Sort kurang efisien dibandingkan Counting Sort dan lebih tepat digunakan sebagai algoritma pembanding dalam penelitian ini.

6.4 Analisis Fibonacci Search

Fibonacci Search digunakan untuk melakukan pencarian data setelah proses pengurutan menggunakan *Counting Sort* selesai dilakukan. Berdasarkan hasil pengujian, algoritma ini mampu menemukan nilai yang dicari dengan jumlah langkah yang relatif sedikit. Program juga menampilkan proses pencarian (trace) sehingga pengguna dapat melihat tahapan pencarian yang dilakukan oleh algoritma.

Fibonacci Search bekerja dengan memanfaatkan deret Fibonacci untuk menampilkan ruang pencarian secara bertahap hingga data ditemukan atau dipastikan tidak terdapat dalam array. Dengan kompleksitas waktu $O(\log n)$, algoritma ini mampu melakukan pencarian secara

efisien pada data yang telah terurut. Oleh karena itu, penggunaan Fibonacci Search setelah *Counting Sort* menghasilkan kombinasi algoritma yang efektif untuk mengolah data nilai ujian mahasiswa.

6.5 Analisis Hasil Eksperimen

Berdasarkan seluruh hasil pengujian yang telah dilakukan, *Counting Sort* menunjukkan performa yang lebih baik dibandingkan *Bubble Sort* pada berbagai ukuran data yang diuji. Perbedaan performa tersebut semakin terlihat ketika jumlah data bertambah. *Counting Sort* mampu mempertahankan waktu eksekusi yang relatif rendah dan stabil, sedangkan *Bubble Sort* mengalami peningkatan waktu eksekusi yang cukup signifikan pada ukuran data yang lebih besar.

Hasil pengujian juga menunjukkan bahwa karakteristik data sangat mempengaruhi kinerja algoritma. Pada studi kasus distribusi nilai ujian mahasiswa dengan rentang nilai 0–100, *Counting Sort* dapat bekerja secara optimal karena rentang data yang digunakan relatif kecil dan tetap. Kondisi tersebut membuat proses pengurutan dapat dilakukan dengan lebih cepat dan efisien dibandingkan *Bubble Sort*, sehingga algoritma ini lebih sesuai untuk digunakan pada data dengan karakteristik serupa.

Selain proses pengurutan yang efisien, Fibonacci Search juga mampu melakukan pencarian nilai dengan baik pada data yang telah terurut. Kombinasi antara *Counting Sort* dan Fibonacci Search menghasilkan sistem yang efektif dalam melakukan pengurutan dan pencarian data. Dengan demikian, kedua algoritma tersebut terbukti sesuai untuk diterapkan pada studi kasus distribusi nilai ujian mahasiswa karena mampu menghasilkan proses pengolahan data yang cepat, efisien, dan akurat.

BAB VII

KESIMPULAN

7.1 Kesimpulan

Implementasi *Counting Sort* modular berbasis C++ berhasil mengurutkan data tanpa perbandingan melalui fase hitung, akumulasi, dan penempatan. Fitur *trace* program terbukti sukses memvisualisasikan dinamika larik frekuensi secara transparan untuk skala kecil. Sementara itu, *Fibonacci Search* berbasis *divide-and-conquer* diaplikasikan secara presisi untuk mempersempit ruang pencarian. Fungsi ini akurat mengembalikan indeks posisi target atau nilai -1 jika data tidak ditemukan.

Pada kasus nilai rentang 0-100, *Counting Sort* terbukti superior dibanding *Bubble Sort* karena memangkas kompleksitas menjadi linear $O(n + k)$. Saat uji *benchmark* 1.000 data acak, *Counting Sort* selesai dalam 0,370 ms, sedangkan *Bubble Sort* membengkak hingga 22,965 ms. Rentang nilai yang sempit ($k = 101$) meminimalkan alokasi memori larik bantu (*count array*). Karakteristik ini membuat pengurutan berjalan linear tanpa dibebani operasi pertukaran (*swap*) berulang.

Eksperimen ini berhasil memetakan karakteristik, kondisi optimal, serta keterbatasan struktural kedua algoritma. *Counting Sort* optimal pada data bulat berrentang sempit (*stable sort*), namun lemah karena *non in-place* yang boros memori jika rentang meluas. Di sisi lain, *Fibonacci Search* efisien sebesar $O(\log n)$ pada data terurut dengan keunggulan operasi hardware yang hanya memakai penjumlahan dan pengurangan. Integrasi kedua algoritma ini terbukti menghasilkan sistem komputasi nilai akademik yang cepat dan efisien.

7.2 Saran

Pengembangan selanjutnya disarankan untuk mengoptimalkan memori dinamis *Counting Sort* agar mampu menangani data dengan rentang nilai luas tanpa memicu pembengkakan memori puncak (*peak memory*). Fungsionalitas program juga perlu ditingkatkan agar mendukung pengurutan multi-kriteria seperti NIM dan nama secara simultan.

Selain itu, antarmuka berbasis teks (*CLI*) sebaiknya dimigrasikan ke bentuk grafis agar lebih intuitif bagi pengguna awam. Terakhir, implementasi *Fibonacci Search* disarankan diuji pada perangkat tertanam untuk memvalidasi efisiensi operasi aritmetika tingkat rendah secara riil.

REFERENSI

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
- Karumanchi, N. (2017). *Data structures and algorithms made easy: Data structures and algorithmic puzzles*. CareerMonk Publications.
- Kelompok 5. (2026). *Laporan proyek kelompok struktur data: Implementasi Counting Sort dan Fibonacci Search*. Program Studi Teknik Informatika, Fakultas Ilmu Komputer. (File referensi: Laporan_Kelompok_CountingSort_FibonacciSearch.pdf).
- Knuth, D. E. (1998). *The art of computer programming, volume 3: Sorting and searching* (2nd ed.). Addison-Wesley.
- Rahmawati, Y., & Setiawan, H. (2026). Analisis perbandingan algoritma sorting terhadap data numerik, karakter, dan string di Python berdasarkan waktu, memori puncak, dan perpindahan data. *JATI (Jurnal Mahasiswa Teknik Informatika)*, 10(2), 3134–3137.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.